

ATM Simulation System

Technical Documentation & Code Walkthrough

Language: Python 3.x

Type: CLI Application

Status: Technical Review

1. OVERVIEW

The **ATM Simulation System** is a lightweight, command-line interface (CLI) application written in Python. It simulates basic automated teller machine workflows, permitting authenticated users to check account balances, make cash deposits, and process cash withdrawals seamlessly.

2. PROGRAM ARCHITECTURE & VARIABLES

The application maintains operational state through simple global and local variables. The primary data models are outlined below:

Variable	Data Type	Scope	Default / Initial Value	Description
balance	Float / Int	Global	10000	Tracks the active user's monetary balance.
correct_pin	String	Global	"220"	Authentication credential required to log in.
pin	String	Local	User Input	User-entered string for credential verification.
choice	String	Loop	User Input	Menu selection identifier ("1", "2", "3", or "4").
amount	Float	Loop	User Input	Monetary amount entered during deposit or withdrawal.

3. CORE IMPLEMENTATION (SOURCE CODE)

Below is the complete implementation of the Python script:

```

# ATM Simulation System

balance = 10000
correct_pin = "220"

print("==== Welcome to ATM =====")

pin = input("Enter your PIN: ")

if pin == correct_pin:
    print("Login Successful!")
else:
    print("Incorrect PIN!")

while True:
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        print("Your balance is:", balance)

    elif choice == "2":
        amount = float(input("Enter amount to deposit: "))
        balance += amount
        print("Amount deposited successfully!")
        print("New Balance:", balance)

    elif choice == "3":
        amount = float(input("Enter amount to withdraw: "))
        if amount <= balance:
            balance -= amount
            print("Please collect your cash.")
            print("New Balance:", balance)
        else:
            print("Insufficient balance!")

    elif choice == "4":
        print("Thank you for using ATM!")
        break

    else:
        print("Invalid choice!")

```

4. FUNCTIONAL FLOW & LOGIC WALKTHROUGH

4.1 Authentication Phase

When executed, the program prompts the user to enter their PIN. An `if-else` statement checks if the entered `pin` matches `correct_pin` ("220"). If invalid, it prints an error and terminates execution immediately.

4.2 Menu Control Loop

Upon successful login, a continuous `while True` loop displays a menu of available actions:

- **1. Check Balance:** Prints the current value stored in `balance`.
- **2. Deposit:** Converts user input to a `float`, adds it to `balance`, and echoes the updated total.
- **3. Withdraw:** Prompts for withdrawal amount and validates against current funds:
 - If `amount <= balance`: Deducts the amount and updates the state.
 - If `amount > balance`: Prints an *"Insufficient balance!"* message without altering state.
- **4. Exit:** Prints a termination greeting and executes a `break` statement to exit the loop.
- **Invalid Option Handling:** Catches any input other than 1-4 and prompts the user to try again.

5. TECHNICAL LIMITATIONS & SUGGESTED ENHANCEMENTS

Key Observations for Refactoring

While the script effectively demonstrates conditional control flow and loops in Python, several production safeguards are recommended.

1. **Input Validation (Robustness):** Entering non-numeric strings (e.g., `"abc"`) during deposit or withdrawal prompts causes an unhandled `ValueError` exception. Implementing `try-except` blocks is highly recommended.
2. **Negative Amount Handling:** The current system allows negative numerical entries (e.g., depositing `-500` decreases balance, or withdrawing `-500` increases balance). Adding strict check conditions like `if amount > 0:` will fix this issue.
3. **Authentication Attempt Limit:** A single wrong PIN entry halts the program. Incorporating a retry counter (e.g., 3 attempts) improves standard user experience.